

```
In [1]: import pandas as pd
```

```
In [2]: import numpy as np
```

```
In [4]: df=pd.read_csv('iris-data - iris-data.csv')
```

```
In [5]: df
```

Out[5]:

	sepal length	sepal width	petal length	petal width	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 5 columns

```
In [6]: df.head()
```

Out[6]:

	sepal length	sepal width	petal length	petal width	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [7]: df.describe()
```

Out[7]:

	sepal length	sepal width	petal length	petal width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
In [9]: df.shape
```

```
Out[9]: (150, 5)
```

```
In [10]: df.isnull().sum()
```

```
Out[10]: sepal length    0
          sepal width    0
          petal length    0
          petal width    0
          class          0
          dtype: int64
```

```
In [11]: X=df.drop(['class'],axis=1)
```

```
In [12]: Y=df[['class']]
```

```
In [13]: X.shape
```

```
Out[13]: (150, 4)
```

```
In [14]: y.shape
```

NameError

Traceback (most recent call last)

Cell In[14], line 1

----> 1 y.shape

NameError: name 'y' is not defined

```
In [15]: Y.shape
```

```
Out[15]: (150, 1)
```

```
In [16]: from sklearn.model_selection import train_test_split
```

```
In [17]: X_train, X_test ,Y_train, Y_test = train_test_split(X,Y,test_size=0.25,random_state=4
```

```
In [18]: X_train.shape
```

```
Out[18]: (112, 4)
```

```
In [25]: from sklearn.tree import DecisionTreeClassifier
```

```
In [26]: dt = DecisionTreeClassifier(criterion = 'entropy')
```

```
In [27]: dt.fit(X_train,Y_train)
```

```
Out[27]: DecisionTreeClassifier
          DecisionTreeClassifier(criterion='entropy')
```

```
In [28]: Y_pred = dt.predict(X_test)
```

```
In [29]: Y_pred
```

```
Out[29]: array(['Iris-virginica', 'Iris-setosa', 'Iris-virginica',  
                'Iris-virginica', 'Iris-virginica', 'Iris-versicolor',  
                'Iris-virginica', 'Iris-setosa', 'Iris-setosa', 'Iris-virginica',  
                'Iris-setosa', 'Iris-setosa', 'Iris-setosa', 'Iris-versicolor',  
                'Iris-virginica', 'Iris-setosa', 'Iris-versicolor', 'Iris-setosa',  
                'Iris-setosa', 'Iris-virginica', 'Iris-setosa', 'Iris-virginica',  
                'Iris-versicolor', 'Iris-setosa', 'Iris-setosa', 'Iris-setosa',  
                'Iris-setosa', 'Iris-setosa', 'Iris-setosa', 'Iris-virginica',  
                'Iris-versicolor', 'Iris-setosa', 'Iris-virginica', 'Iris-setosa',  
                'Iris-versicolor', 'Iris-virginica', 'Iris-virginica',  
                'Iris-versicolor'], dtype=object)
```

```
In [30]: dt.score(X_test, Y_test)
```

```
Out[30]: 0.9736842105263158
```

```
In [37]: dt.feature_importances_
```

```
Out[37]: array([0.          , 0.0156062 , 0.07501716, 0.90937664])
```

```
In [41]: from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1_score
```

```
In [52]: confusion_matrix(Y_test, Y_pred)
```

```
Out[52]: array([[18,  0,  0],  
                [ 0,  7,  1],  
                [ 0,  0, 12]], dtype=int64)
```

```
In [53]: precision_score(Y_test, Y_pred, average='weighted')
```

```
Out[53]: 0.9757085020242916
```

```
In [58]: recall_score(Y_test, Y_pred, average='weighted')
```

```
Out[58]: 0.9736842105263158
```

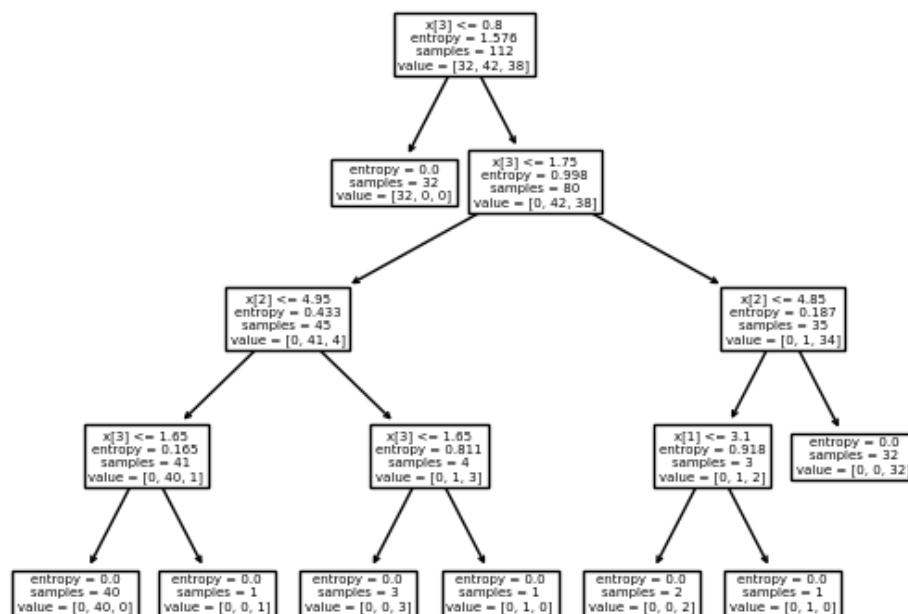
```
In [59]: f1_score(Y_test, Y_pred, average='weighted')
```

```
Out[59]: 0.9733333333333335
```

```
In [72]: from sklearn import tree
```

In [73]: `tree.plot_tree(dt)`

Out[73]: [Text(0.5, 0.9, 'x[3] <= 0.8\nentropy = 1.576\nnsamples = 112\nvalue = [32, 42, 38]'),
Text(0.4230769230769231, 0.7, 'entropy = 0.0\nnsamples = 32\nvalue = [32, 0, 0]'),
Text(0.5769230769230769, 0.7, 'x[3] <= 1.75\nentropy = 0.998\nnsamples = 80\nvalue = [0, 42, 38]'),
Text(0.3076923076923077, 0.5, 'x[2] <= 4.95\nentropy = 0.433\nnsamples = 45\nvalue = [0, 41, 4]'),
Text(0.15384615384615385, 0.3, 'x[3] <= 1.65\nentropy = 0.165\nnsamples = 41\nvalue = [0, 40, 1]'),
Text(0.07692307692307693, 0.1, 'entropy = 0.0\nnsamples = 40\nvalue = [0, 40, 0]'),
Text(0.23076923076923078, 0.1, 'entropy = 0.0\nnsamples = 1\nvalue = [0, 0, 1]'),
Text(0.46153846153846156, 0.3, 'x[3] <= 1.65\nentropy = 0.811\nnsamples = 4\nvalue = [0, 1, 3]'),
Text(0.38461538461538464, 0.1, 'entropy = 0.0\nnsamples = 3\nvalue = [0, 0, 3]'),
Text(0.5384615384615384, 0.1, 'entropy = 0.0\nnsamples = 1\nvalue = [0, 1, 0]'),
Text(0.8461538461538461, 0.5, 'x[2] <= 4.85\nentropy = 0.187\nnsamples = 35\nvalue = [0, 1, 34]'),
Text(0.7692307692307693, 0.3, 'x[1] <= 3.1\nentropy = 0.918\nnsamples = 3\nvalue = [0, 1, 2]'),
Text(0.6923076923076923, 0.1, 'entropy = 0.0\nnsamples = 2\nvalue = [0, 0, 2]'),
Text(0.8461538461538461, 0.1, 'entropy = 0.0\nnsamples = 1\nvalue = [0, 1, 0]'),
Text(0.9230769230769231, 0.3, 'entropy = 0.0\nnsamples = 32\nvalue = [0, 0, 32]')]



In [74]: `from sklearn.model_selection import cross_val_score`

In [75]: X

Out[75]:

	sepal length	sepal width	petal length	petal width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

In [77]: Y

Out[77]:

	class
0	Iris-setosa
1	Iris-setosa
2	Iris-setosa
3	Iris-setosa
4	Iris-setosa
...	...
145	Iris-virginica
146	Iris-virginica
147	Iris-virginica
148	Iris-virginica
149	Iris-virginica

150 rows × 1 columns

In [78]: dt.clf = DecisionTreeClassifier(random_state = 0)

In [79]: cv_score = cross_val_score(dt.clf, X, Y, cv = 5)

In [80]: print ("Score of each fold:", cv_score)

Score of each fold: [0.96666667 0.96666667 0.9 0.96666667 1.]

In [81]: mean_score = cv_score.mean()

```
In [82]: print("Mean accuracy:", mean_score)
```

Mean accuracy: 0.9600000000000002

```
In [83]: std_dev = cv_score.std()
```

```
In [84]: print("Standard Deviation:" , std_dev)
```

Standard Deviation: 0.03265986323710903

```
In [ ]:
```